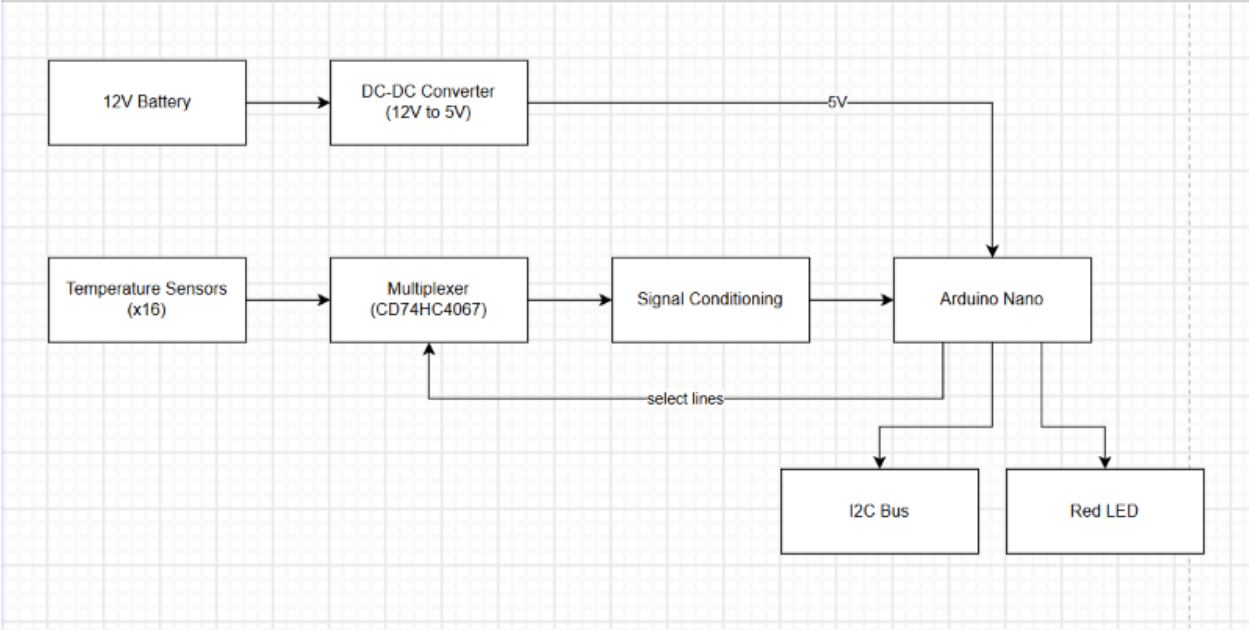
NMP Design Doc

Oct 12, 2026 · @Irei Hashimoto

1. Overview

This system monitors battery-cell temperatures, reports the hottest reading over I2C, and flags a fault above 60°C. Each temperature sensor produces a voltage that falls as temperature rises. A multiplexer connects one sensor at a time to a signal-conditioning stage, and the Arduino Nano's 10-bit ADC converts that voltage to a number. The firmware converts each reading to a temperature, finds the maximum across all channels, and sends it over I2C. A blinking red LED shows the system is running and indicates a fault. A 12 V to 5 V DC-DC converter powers the board from the car's low-voltage battery.

Figure 1 Block Diagram



2. Requirements and assumptions

The project requires a circuit that reads 16 temperature sensors, converts them to digital values, and reports the highest temperature over I2C.

Requirements

- 16 temperature sensors
- A CD74HC4067 analog multiplexer (schematic); CD4051BE in physical build
- A signal conditioning stage between the multiplexer and the Nano
- An Arduino Nano that converts each reading with its ADC and finds the highest temperature
- I2C output of that temperature
- A blinking red LED, with a fault indicated above 60°C
- A 12 V to 5 V DC-DC converter (schematic)

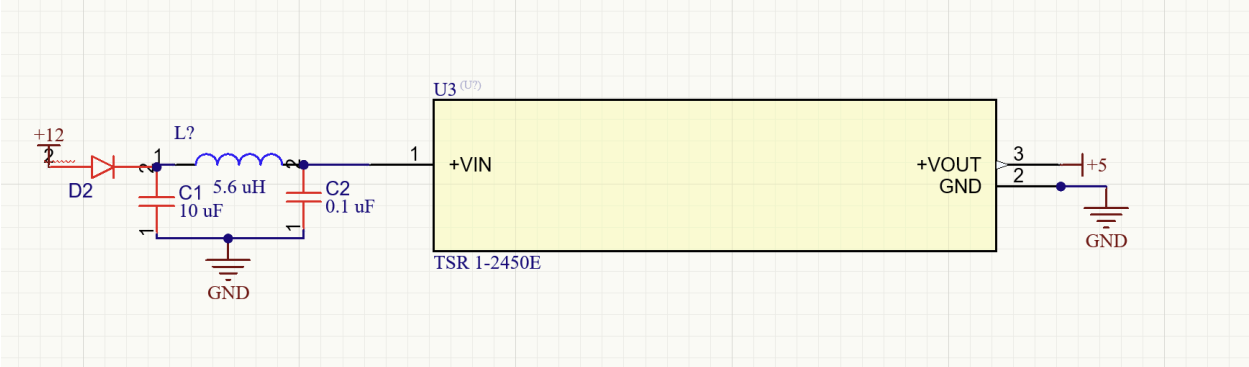
Assumptions

- Sensor supply: 5V through 680 Ω, per the datasheet test circuit.
- LV battery voltage range: fill in from the leads' answer, or state the range you assumed.
- Number of sensors in the physical build (8 or 16): fill in from the leads' answer.
- ADC reference: default 5 V; conversion formula uses 1024.

3. Hardware

One short subsection per block, in signal order. Target: 2–3 pages plus schematic crops. Each subsection gets a cropped schematic figure.

3.1 Power



- **Current budget table:**

Load	Current (mA)	Source

Sensors (worst case, 120°C)	16mA
Arduino Nano	50mA
Mux	8 µA
LED	232 mA
I2C pull-ups	3.3 mA
Total: ~300mA	

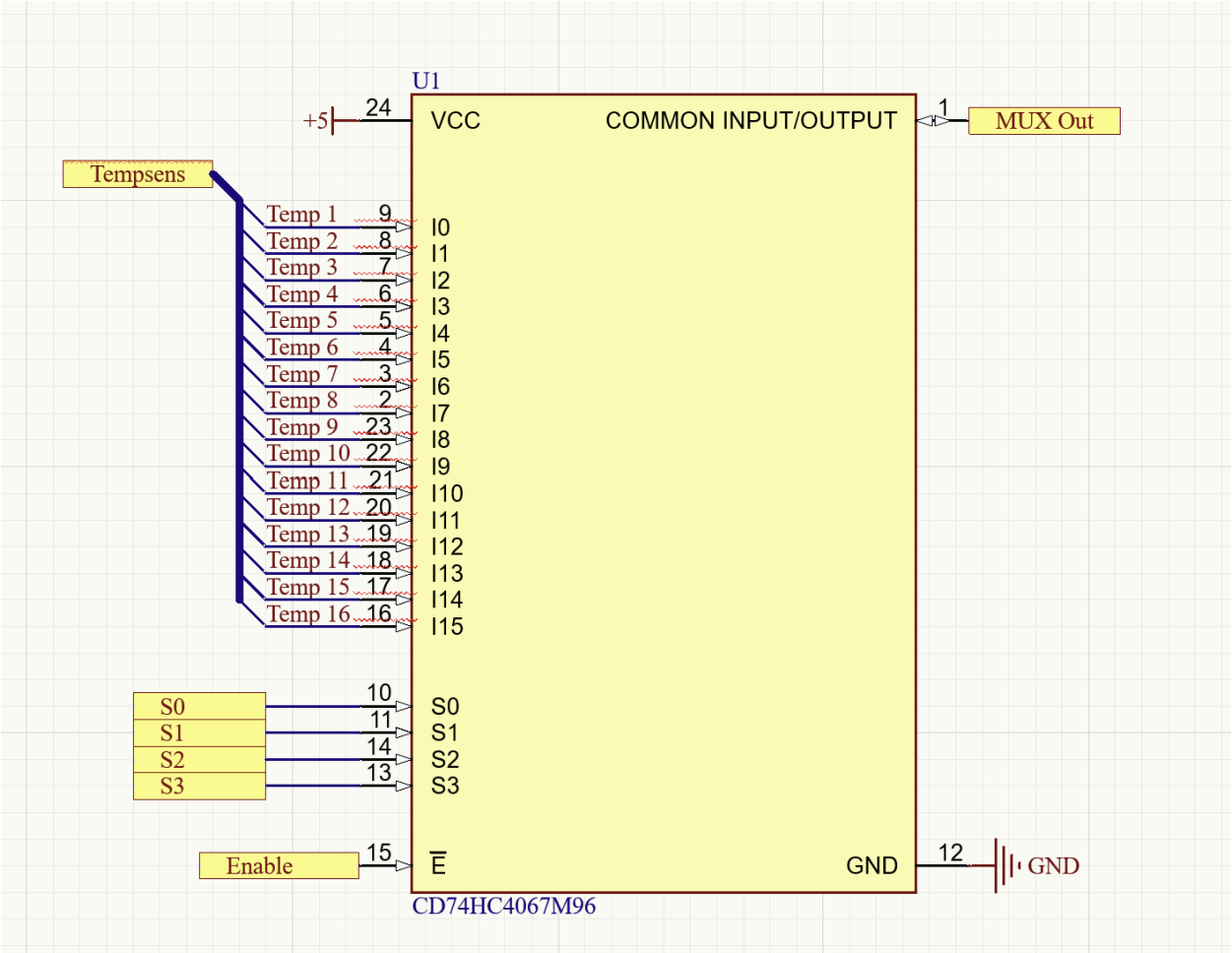
3.2 Temperature sensors

- Each sensor is in series with a 680 Ω resistor between the 5 V rail and ground, which matches the test circuit in the sensor datasheet [1]. We match it because the datasheet's voltage-to-temperature table is only valid under those conditions.
- The resistor and sensor form a voltage divider, and the measured signal is the voltage at the point between them. As the sensor heats, its effective resistance falls, so its share of the 5 V falls. The sensor's voltage therefore drops as temperature rises.

Table 1: Sensor output at key temperatures (from datasheet Table 5 [1])

Temperature (°C)	Sensor voltage (V)	ADC reading (5 V ref)
-40	2.44	499
0	2.17	444
25	1.86	380
60	1.51	309
120	1.3	266

3.3 Multiplexer



- MUX selects a channel from the binary code sent from the Arduino. It follows this truth table:

Table 4-1. Truth Table

S0	S1	S2	S3	$\overline{E}$	SELECTED CHANNEL
X	X	X	X	1	None
0	0	0	0	0	0
1	0	0	0	0	1
0	1	0	0	0	2
1	1	0	0	0	3
0	0	1	0	0	4
1	0	1	0	0	5
0	1	1	0	0	6
1	1	1	0	0	7
0	0	0	1	0	8
1	0	0	1	0	9
0	1	0	1	0	10
1	1	0	1	0	11
0	0	1	1	0	12
1	0	1	1	0	13
0	1	1	1	0	14
1	1	1	1	0	15

- The Mux's Enable pins are also connected to the microcontroller, however, this is held low as this sets the IC as a multiplexor.
- The MUX is also supplied with 5v as is acceptable within the 2-6v range on the datasheet.
- The mux has a ON resistance of about 160 ohms at worst and 70 ohm typical
- Settling delay can be calculated with the equation:

  Settling delay = N(R(source) + R(on)) x C(total) = ~1 ns

  N = 6.93 time constants (τ)

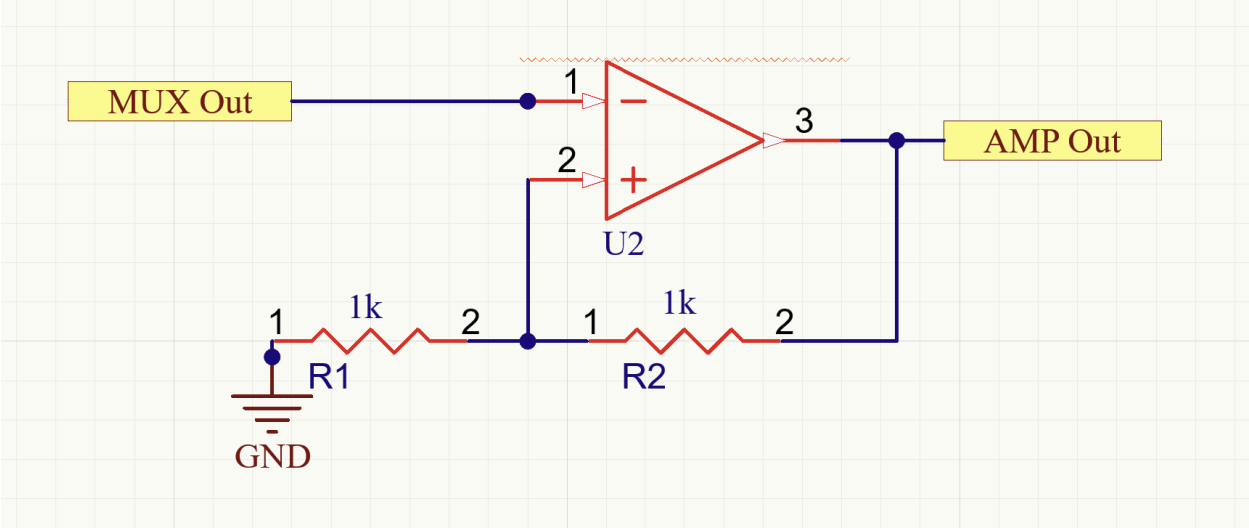
  Rsource = 670 Ohms

  Ron = 160 Ohms

  C 56 pF nano

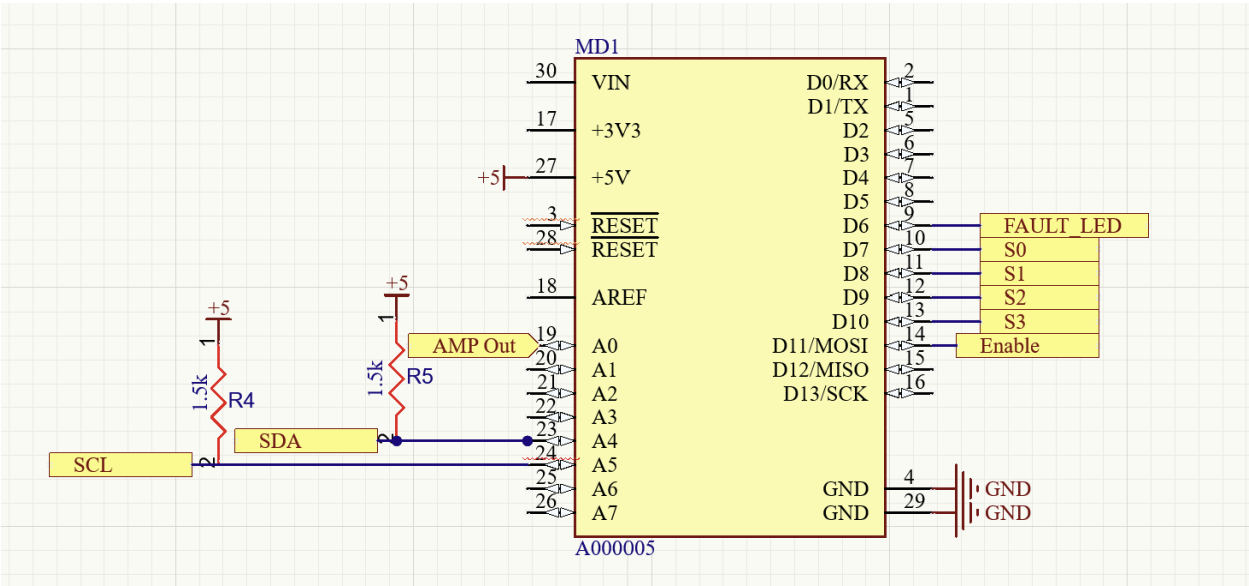
- Despite this low estimated settling time, non-idealities from the breadboard could make this settling delay longer. To solve the issue of reading this settling delay a software delay of 1 microsecond is added.

3.4 Signal conditioning



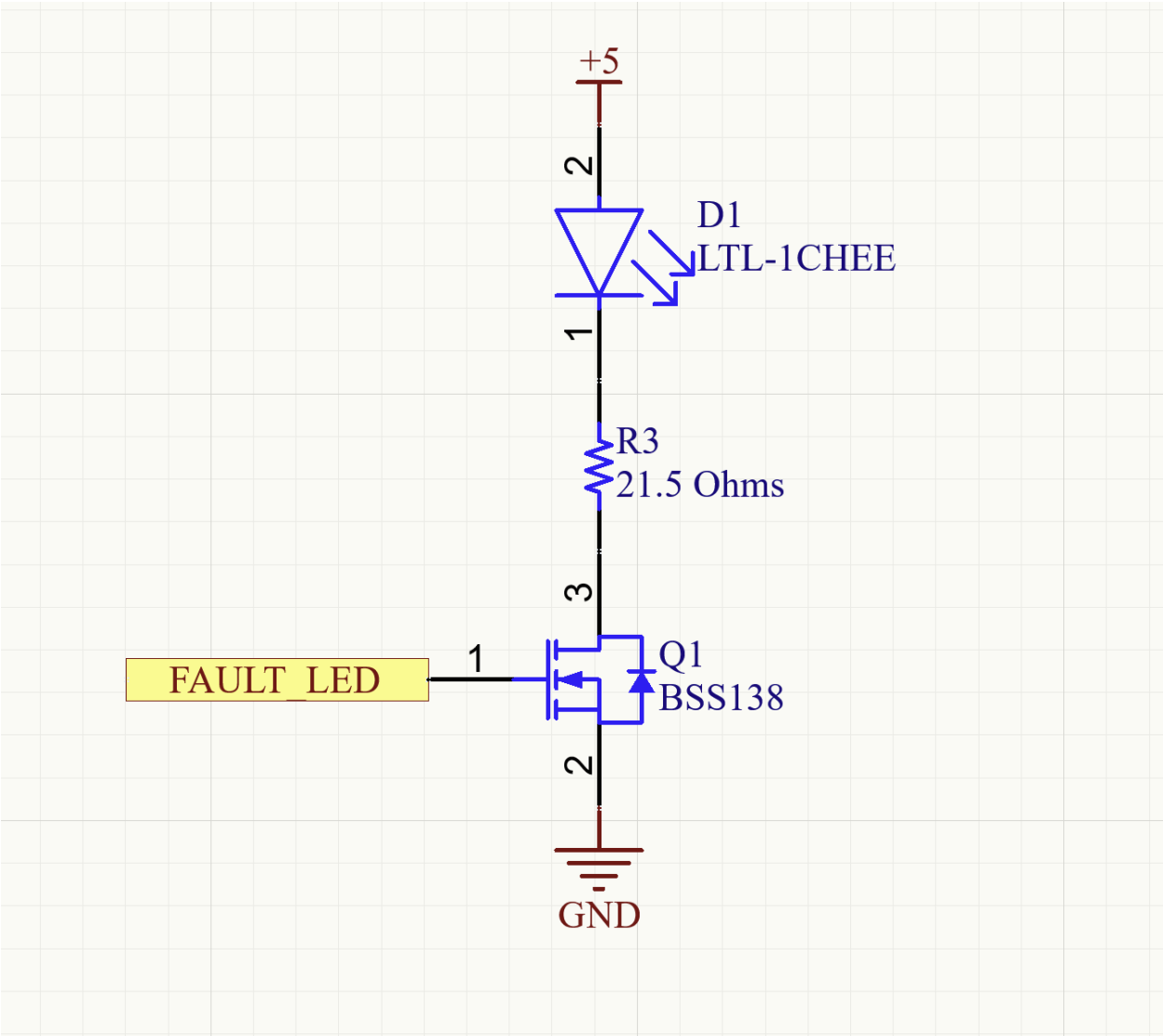
- The sensors would produce a voltage from 0-~2.5V, however our Arduino nano is set for a 5v range. This creates a signal that isn't as accurate from the lower resolution.
- To solve this there is an non-inverting Operational Amplifier that amplifies voltage to a gain of 2.

3.5 Nano and ADC



- ADC: 10-bit, 5 V reference

3.6 Fault LED



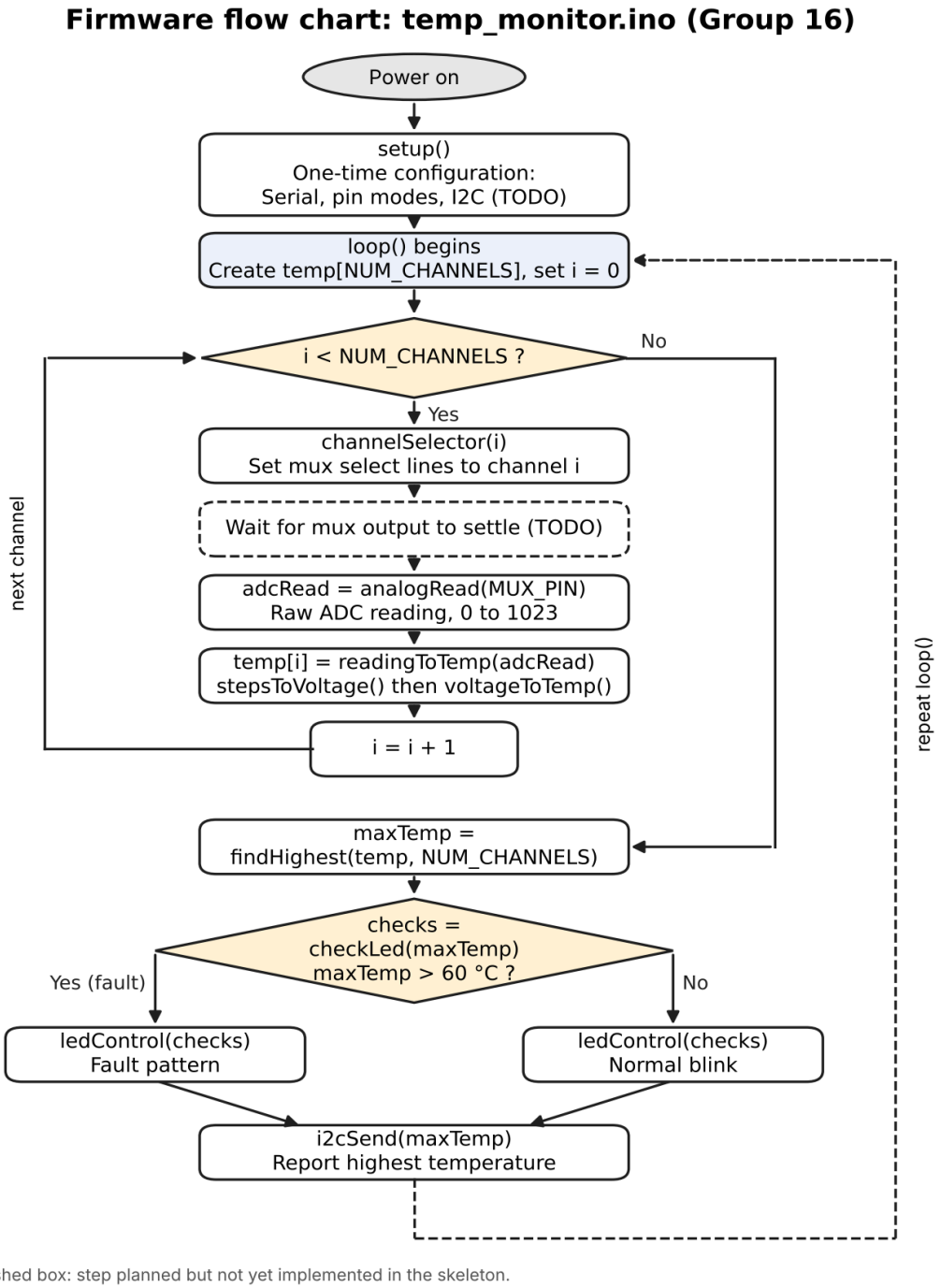
3.7 I2C

- The temperature that the Arduino nano produces is then emitted via I2C to other boards
- The I2C circuit is pulled up to 1.5k ohms. This is because the lower the resistance the less RLC noise would be caused from the added impedance to the circuit.

4. Software

The firmware (software/temp_monitor/temp_monitor.ino) runs on the Arduino Nano. Each pass of loop() reads every channel, converts each reading to a temperature, finds the maximum, checks it against the 60 °C threshold, updates the LED, and sends the maximum over I2C.

Figure 2: Firmware flow chart



Dashed box: step planned but not yet implemented in the skeleton.

4.1 Program structure

setup() runs once to configure the serial port, the output pins (mux select lines and LED) and I2C. loop() then repeats indefinitely:

1. For each channel i from 0 to NUM_CHANNELS − 1: select channel i on the mux, read the analog pin with analogRead(), convert the reading to a temperature, and store it in slot i of the temperature array(1d Array)
    a. I thought about using a 2d array but I realized having int and float in same array wouldn't work.
2. Find the highest temperature in the array.
3. Determine whether it exceeds 60 °C.
4. Update the LED (normal blink, or fault pattern).
5. Send the highest temperature over I2C.

Functions:

- channelSelector(channel): sets the MUX select lines to the binary value of the channel. Returns nothing.
- readingToTemp(reading): converts an ADC reading to °C by calling stepsToVoltage, then voltageToTemp
- findHighest(temperatures, count): returns the highest temperature in array
- checkLed(temperature): returns true if the temperature in argument is above 60 °C.
- ledControl(fault): Controls LED. Returns nothing.
- i2cSend(temperature): sends the highest temperature over I2C. Returns nothing.
- analogRead(pin): built-in Arduino function [3]. Reads the voltage on the given analog pin (the mux output, `MUX_PIN`) with the 10-bit ADC and returns a value from 0 to 1023.

  Helper Functions

- stepsToVoltage(reading): converts an ADC reading to volts: V = reading × V_ref ÷ 1024.
- voltageToTemp(voltage): converts volts to °C using the datasheet table.

4.2 Design decisions

- One value per conversion function. Each conversion helper takes one value and returns one value, so each can be tested against hand calculations: for example, a reading of 400 should give 1.9531 V and 17.63 °C. loop() stores each result in the array.
- Sensor table as a constant. The 33 datasheet voltages (−40 °C to 120 °C in 5 °C steps) are stored once as a const array. Temperatures are evenly spaced, so a table index converts directly to a temperature and only voltages need storing.[1]
- Channel count as a single constant.[4,5] NUM_CHANNELS sizes the array and bounds the loop. It is 16 for the CD74HC4067 schematic and 8 for the CD4051BE build, so switching requires one edit. The CD4051BE also uses 3 select lines instead of 4, so the select-line pin constants change with it.
- Temperature array passed with its count. findHighest receives the array and the count as arguments rather than reading a global, so it works for any channel count and can be tested with made-up arrays.
- The sensor is non-linear, and the datasheet recommends linear interpolation between table values for reasonable accuracy, so I just used that.

- I thought about using a 2d array but realized that temperature is evenly spaced so there was no need to store.
- ADC conversion uses 1024,[2] matching the ATmega328P datasheet's conversion formula. The difference from 1023 is about 0.1%.
- MUX settling. After switching channels, the firmware waits briefly before reading so the output reflects the newly selected sensor. The delay is sized during the build phase.

4.3 Math Explanation

Step 1: ADC reading to voltage: The Nano's ADC is 10-bit, so the reading is numbered from 0 to 1023, with a 5 V reference, so V = reading × 5 / 1024 One count (LSB) is 5 / 1024 = 4.88 mV.[2]

Step 2: Voltage to temperature (linear interpolation) The sensor table lists voltages from −40 °C to 120 °C in 5 °C steps. Table index k corresponds to: T(k) = −40 + 5k The firmware finds the two entries the voltage falls between, V(k) ≥ V > V(k+1), and interpolates linearly: fraction = (V(k) − V) / (V(k) − V(k+1)) T = T(k) + 5 × fraction Because the sensor's voltage falls as temperature rises, V(k) is the higher voltage and T(k) the lower temperature.

Worked example 1:

- Reading 400 V = 400 × 5 / 1024 = 1.9531 V
- 15 °C (1.99 V) and 20 °C (1.92 V) fraction = (1.99 − 1.9531) / (1.99 − 1.92) = 0.0369 / 0.07 = 0.527
- 15 + 5 × 0.527 = 17.63 °C (no fault**)**

4.4 Still to do for the build phase

- Implement and test against calculations
- Handle readings outside the table range (below 1.30 V or above 2.44 V)

5. Testing and limitations

No build yet

6. References

"AI tools were used for planning, research, learning, drafting, and support; all code was written by the authors."

1. enepaq, *Li-ion building block with Sony/Murata VTC5A*, datasheet, rev. E, Jan. 2025. Table 5: sensor voltage vs. temperature.
2. Microchip Technology, *ATmega328P*, datasheet. Analog-to-Digital Converter chapter: conversion result formula.

3. Arduino, *Language Reference*: `analogRead()`, `digitalWrite()`, `pinMode()`, `bitRead()`, `millis()`.
4. Texas Instruments, *CD74HC4067, CD74HCT4067 16-Channel Analog Multiplexer/Demultiplexer*, datasheet SCHS209D, Dec. 2024.
5. Texas Instruments, *CD4051B, CD4052B, CD4053B CMOS Analog Multiplexers/Demultiplexers*, datasheet SCHS047L, Sep. 2023.